

Non-Graded Project: Deploy Production LLM Platform: TechUnicorn's Enterprise Solution

Description

Design and implement a complete production-grade LLM platform for TechUnicorn, a startup competing with OpenAI and Anthropic. Apply all course skills to build automated CI/CD pipelines, implement enterprise security controls, and configure monitoring and cost optimization systems that meet investor requirements for 99.99% uptime and \$300K monthly operational budget.

Learning Objectives

By the end of this project, learners will be able to:

- Design and implement automated CI/CD pipelines for multi-model LLM infrastructure with blue-green deployments.
- Apply comprehensive security controls, including API protection, prompt injection defense, and compliance-ready audit logging.



- Configure production monitoring, auto-scaling, and cost optimization to meet enterprise SLA and budget requirements.
- Synthesize all course learnings into a cohesive, production-ready platform architecture.
- Create professional documentation suitable for technical and executive stakeholders.



Assignment Components

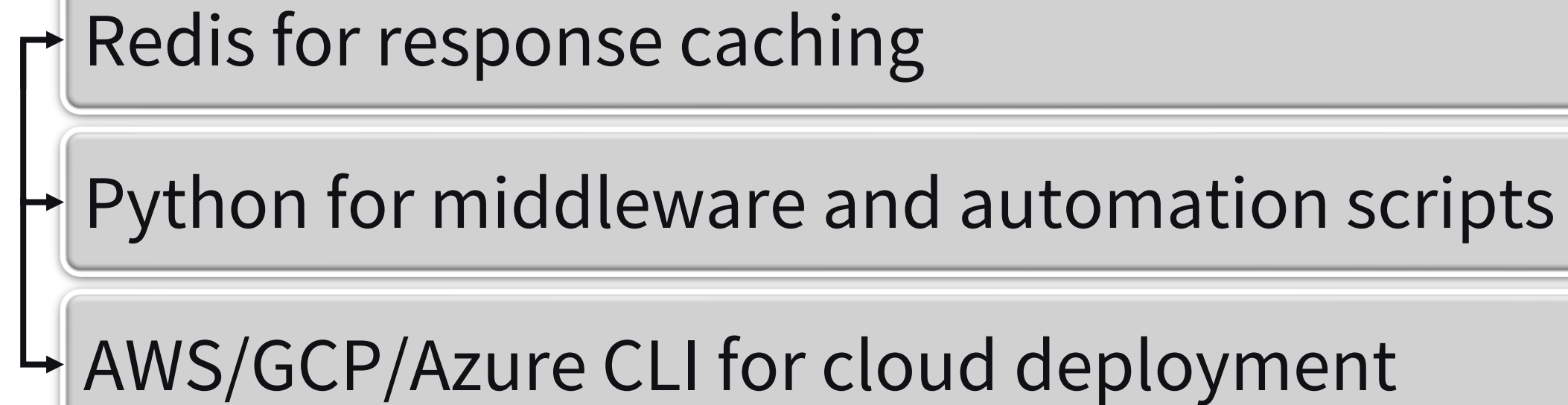
Introductory Slide

Title: Mission Critical: Launch TechUnicorn's Enterprise LLM Platform

Subtitle: Your platform engineering expertise determines whether TechUnicorn can compete in the \$50B enterprise AI market

Tools to be used:

- Docker and Docker Compose for containerization
- Kubernetes (Minikube or cloud provider) for orchestration
- Terraform for infrastructure as code
- GitHub Actions for CI/CD automation
- API Gateway (Kong or AWS API Gateway) for security
- Prometheus and Grafana for monitoring
- HashiCorp Vault for secrets management

- 
- Redis for response caching
 - Python for middleware and automation scripts
 - AWS/GCP/Azure CLI for cloud deployment

Scenario

You've been hired as Lead Platform Engineer by TechUnicorn, a startup launching an enterprise LLM platform to compete with OpenAI and Anthropic. The platform must support multiple LLM models (GPT-4, Claude, Llama), serve 100 enterprise clients, and handle 20 million API requests daily with 99.99% uptime. With a \$500K monthly budget and recent competitor breaches making security a top priority, you must deliver enterprise-grade infrastructure at 40% lower operational costs than competitors.

Instructions

Instructions

Review Requirements and Download Starter Files

- Read through all five requirements carefully. Download the starter files package, which contains Terraform templates, Kubernetes manifests, and Python scaffolding for security and monitoring components.
- **Expected Outcome:** You have a clear understanding of all deliverables and have extracted the starter files to your working directory.
- **Tip:** Create a checklist to track completion of each requirement component.

Set Up Infrastructure Foundation

→ Complete the Terraform configuration by defining your cloud provider, VPC networking, and compute resources. Configure the Kubernetes manifests with appropriate resource requests, limits, and health probes for LLM workloads.

→ **Expected Outcome:** Running 'terraform plan' shows a valid execution plan with no errors. Kubernetes manifests pass 'kubectl apply --dry-run=client' validation.

→ **Warning:** Ensure you're using a test/dev environment to avoid unexpected cloud charges. Set billing alerts before deploying.

Build CI/CD Pipeline

- Create the GitHub Actions workflow with stages for building Docker images, running Trivy security scans, and deploying with a blue-green strategy. Configure rollback triggers based on health check failures.
- **Expected Outcome:** A successful pipeline run that builds, scans (with zero CRITICAL findings), and deploys to your target environment with health check validation passing.
- **Tip:** Test the pipeline incrementally—first get the build stage working, then add scanning, then deployment.

Implement Security Controls

→ Complete the API Gateway configuration with authentication and rate limiting. Implement the prompt injection validator with pattern detection. Set up the compliance audit logger with all required fields for SOC 2 and GDPR.

→ **Expected Outcome:** The prompt validator correctly blocks test injection attempts from the provided test suite. Audit logs capture all required fields with tamper-evident hashing.

→ **Warning:** Never commit genuine API keys or secrets to your repository. Use environment variables or secrets management.

Configure Monitoring and Optimization

→ Set up Prometheus with custom LLM metrics and configure Grafana dashboards. Implement the HPA configuration with custom metrics. Complete the cost optimizer with semantic caching and model routing logic.

→ **Expected Outcome:** Grafana dashboards display real-time metrics. HPA scales pods based on custom metrics. Cost optimizer demonstrates cache hits and appropriate model routing decisions.

→ **Tip:** Use the provided test prompts to verify caching behavior—repeated similar queries should show cache hits.

Create Professional Documentation

- Write the executive summary highlighting business value and key capabilities.
Create architecture diagrams showing system components and data flow.
Document the security posture, operational procedures, and cost projections.
- **Expected Outcome:** Documentation package is clear, professional, and suitable for both technical teams and executive stakeholders. Diagrams accurately reflect the implemented architecture.
- **Tip:** Use tools like draw.io or Lucidchart for architecture diagrams. Keep the executive summary focused on business outcomes, not technical details.

Test, Validate, and Submit

Run end-to-end tests to verify all components work together. Validate that the platform meets the success criteria (99.99% uptime capability, security controls passing, and cost targets achievable). Package all deliverables for submission.

Expected Outcome: All five deliverables are complete, tested, and packaged. The platform demonstrates enterprise-grade reliability, security, and cost efficiency.

Warning: Remember to clean up cloud resources after testing to avoid ongoing charges.

Compile Your Findings

1 Requirement 1: Complete Infrastructure Code

Description: Terraform templates, Kubernetes manifests, and Docker configurations for the entire platform with multi-region deployment capability

Include:

- Optimized Dockerfiles with multi-stage builds for LLM inference servers
- Kubernetes deployment manifests with resource limits and health checks
- Terraform modules for compute, networking, and storage resources
- Environment-specific variable files (dev, staging, prod)
- Documentation of infrastructure components and their scaling characteristics

2 Requirement 2: CI/CD Pipeline Implementation

Description: Full automation from code commit to production deployment, including testing, security scanning, and rollback procedures

Include:

- GitHub Actions workflow YAML with all stages (build, test, scan, deploy)
- Trivy security scan configuration and sample results
- Blue-green deployment scripts with health check validation
- Automated rollback triggers and procedures
- Pipeline execution time analysis (target: <30 minutes)

3 Requirement 3: Security and Compliance Package

Description: API security configuration, prompt injection defenses, audit logging, and compliance documentation for SOC 2 and GDPR

Include:

- API Gateway configuration with OAuth 2.0 and rate limiting rules
- IAM policy documents demonstrating least-privilege access
- Prompt injection defense middleware with test results
- Audit logging schema and sample compliance queries
- Compliance matrix mapping controls to SOC 2 and GDPR requirements

4 Requirement 4: Monitoring and Operations Suite

Description: Complete observability stack with dashboards, alerts, auto-scaling, and cost optimization, demonstrating the platform can meet SLA and budget targets

Include:

- Prometheus configuration with custom LLM metrics
- Grafana dashboard JSON covering performance, availability, usage, and cost
- Alert rules with escalation procedures
- Kubernetes HPA configuration with queue depth triggers
- Cost optimization analysis showing projected savings (target: 40% reduction)

Runbook documentation for incident response and capacity planning

5 Requirement 5: Professional Documentation Package

Description: Executive-ready documentation summarizing the platform architecture, security posture, and operational capabilities for technical and executive stakeholders.

Include:

- Executive summary (1 page) suitable for board presentation
- Technical architecture documentation with diagrams
- Security posture summary with compliance status
- Operational runbook with escalation procedures

Cost analysis with projected ROI

Summary

This capstone project provides hands-on experience with enterprise LLM platform engineering through the practical application of automated deployment, security implementation, and production operations. Participants have developed competency in CI/CD pipeline design, security framework implementation, and monitoring/cost optimization, which are directly applicable to MLOps, platform engineering, and DevOps roles at companies deploying production AI systems.

Key Takeaways:

- Automated CI/CD pipelines with blue-green deployments enable reliable, zero-downtime LLM updates.
- Layered security controls (API protection, prompt injection defense, audit logging) are essential for enterprise compliance.
- Comprehensive monitoring with intelligent auto-scaling and cost optimization enables platforms to meet demanding SLA and budget requirements.

Resources

Primary Materials (link to files: [Course-End-Project-StarterFiles.zip](#))

- Starter repository with sample LLM application code
- Terraform module templates
- Kubernetes manifest templates
- Compliance requirements matrix
- Security test cases (injection tests JSON)

Reference Documentation

- Docker Documentation: <https://docs.docker.com/>
- Kubernetes Documentation: <https://kubernetes.io/docs/>
- Terraform Documentation: <https://developer.hashicorp.com/terraform>
- GitHub Actions Documentation: <https://docs.github.com/en/actions>
- Prometheus Documentation: <https://prometheus.io/docs/>
- Grafana Documentation: <https://grafana.com/docs/>
- OWASP Top 10 for LLM Applications: <https://genai.owasp.org/llm-top-10/>

Solution/Exemplar

→ Course_End_Project_Solution (available in starter files package)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.